

簡化主要資料管理作業：運用生成式 AI 比對及合併資料！

來源：<https://codelabs.developers.google.com/mdm-with-gemini?hl=zh-tw>

步驟數：12

在本程式碼研究室中，我們將說明 Gemini 1.0 Pro 如何簡化 BigQuery 公開資料集中可用的 citibike_stations 資料，以及充實和簡化等主資料管理應用程式。

目錄

1. [總覽](#)
2. [目標](#)
3. [需求條件](#)
4. [事前準備](#)
5. [建立 BigQuery 資料集和外部連線](#)
6. [部署 Gemini 函式呼叫功能 \(Java Cloud Function\)](#)
7. [嘗試解決方法](#)
8. [建立資料表並擴充地址資料](#)
9. [為地址資料生成嵌入](#)
10. [執行向量搜尋，標記重複的地址](#)
11. [清除所用資源](#)
12. [恭喜](#)

1. 總覽

什麼是主要資料管理？

主要資料管理 (MDM) 的目標是為貴機構最重要資料建立單一可靠的資料來源。想像一下，在精心整理的圖書館中，每本書 (資料點) 都正確標示、內容最新，而且容易找到。

主資料代表對公司營運至關重要的核心基礎業務實體。主資料的主要元素如下：

- **業務實體**：例如顧客、產品、供應商、地點和員工等實體，這些都是貴商家業務的核心名詞
- **ID**：確保每個實體在系統中都是獨一無二且可追蹤的專屬 ID
- **屬性**：描述每個實體的特徵，例如顧客地址、產品價格等。

為協助您進一步瞭解主資料，我們將其與交易資料進行比較。交易資料會擷取個別事件 (例如購買或出貨)。而主資料則會定義相關實體，為這些事件提供背景資訊。舉例來說，銷售交易會連結至客戶、產品和銷售人員的主要資料。

雖然導入強大的 MDM 對於策略決策至關重要，但這項作業可能相當複雜，且需要大量資源。這時，生成式 AI 的強大力量就能派上用場，特別是 Gemini 1.0 Pro、Gemini 1.0 Pro Vision 和 Gemini 1.5 Pro 等模型。

2. 目標

在本程式碼研究室中，您將示範如何使用 Gemini 1.0 Pro，簡化主資料管理應用程式 (例如擴充和重複資料刪除)，以處理 BigQuery 公開資料集中的 citibike_stations 資料。

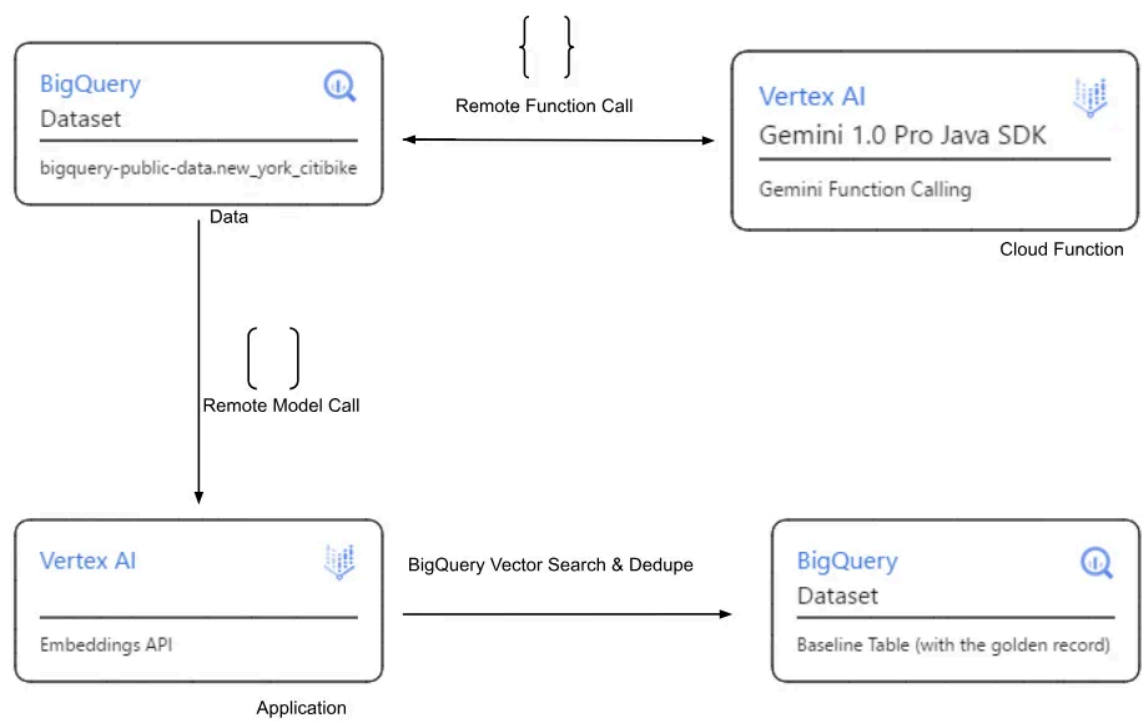
使用項目

- 1. BigQuery 公開資料集 `bigquery-public-data.new_york_citibike`。
- 2. Gemini 函式呼叫 (Java Cloud 函式，可使用[反向 Geocoding API](#) 取得 citibike_stations 資料提供的座標地址資訊)。
- 3. 使用 Vertex AI Embeddings API 和 BigQuery 中的向量搜尋功能找出重複內容。

建構項目

- 1. 您將為這個用途建立 BigQuery 資料集。您將在這個資料集中，使用公開資料集資料表 `bigquery-public-data.new_york_citibike.citibike_stations` 中的資料建立登陸資料表。
- 2. 您將部署 Cloud Functions，其中包含用於地址標準化的 Gemini 函式呼叫功能。
- 3. 您會將經過擴充的地址資料儲存在登陸資料表中 (來自這項示範提供的兩個來源)。
- 4. 您將從 BigQuery 叫用 Vertex AI Embeddings API，處理地址資料。
- 5. 您將使用 BigQuery 向量搜尋功能找出重複記錄。

下圖說明實作時的資料流和步驟。



步驟 3 · 約 0 分鐘

3. 需求條件

- [Chrome](#) 或 [Firefox](#) 瀏覽器
 - 已啟用計費功能的 Google Cloud 專案。
-

4. 事前準備

1. 在 [Google Cloud 控制台](#) 的專案選取器頁面中，選取或建立 Google Cloud [專案](#)。
2. 確認 Cloud 專案已啟用計費功能。瞭解如何[檢查專案是否已啟用計費功能](#)。
3. 您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境，預先載入了 bq。點選 Google Cloud 控制台頂端的「啟用 Cloud Shell」。



4. 連至 Cloud Shell 後，請使用下列指令確認驗證已完成，專案也已設為獲派的專案 ID：

```
gcloud auth list
```

5. 在 Cloud Shell 中執行下列指令，確認 gcloud 指令已瞭解您的專案。

```
gcloud config list project
```

6. 如果未設定專案，請使用下列指令來設定：

```
gcloud config set project <YOUR_PROJECT_ID>
```

7. 前往 [Gemini for Google Cloud Marketplace](#) 啟用 API。您也可以 Cloud Shell 終端機中使用下列指令：

```
gcloud services enable clouddaicompanion.googleapis.com --project PROJECT_ID
```

8. 確認已啟用 BigQuery、BigQuery Connection、Cloud Function、Cloud Run、Vertex AI 和 Cloud Build API。您也可以透過控制台 (使用[這個連結](#)) 執行 gcloud 指令。

如要瞭解 gcloud 指令和用法，請參閱[說明文件](#)。

步驟 5 · 約 0 分鐘

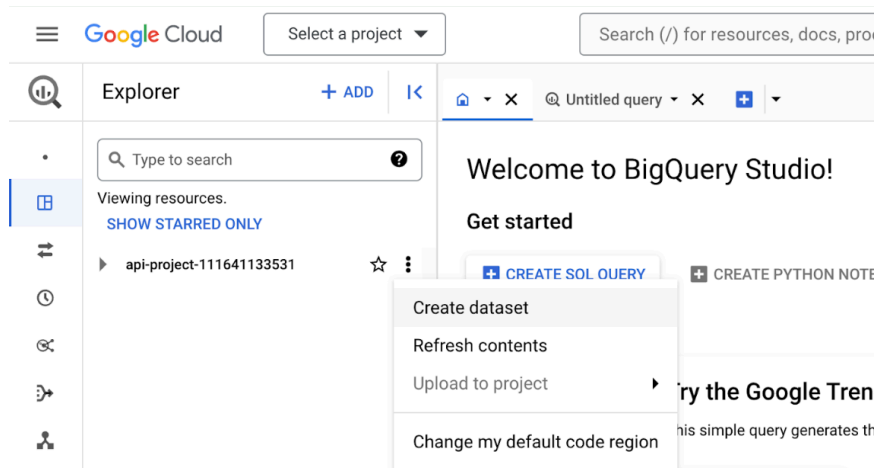
5. 建立 BigQuery 資料集和外部連線

首先，請建立資料集和 Cloud 資源連結。

BigQuery 中的[資料集](#)是應用程式所有資料表和物件的容器。

如要建立資料集，請按照下列步驟操作：

1. 前往 Google Cloud 控制台的「BigQuery」[頁面](#)。
2. 在「Explorer」面板中，選取要建立資料集的專案。
3. 展開「動作」選項 (垂直刪節號圖示)，然後點選「建立資料集」。

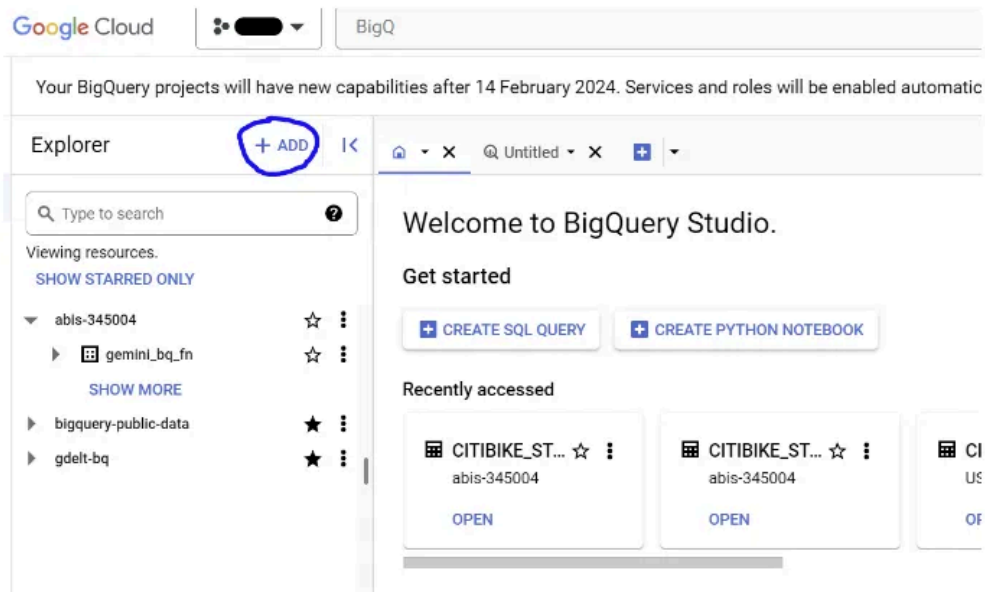


4. 在「Dataset ID」欄位中輸入 `mdm_gemini`。
5. 將位置類型設為 `Multi-region`，並接受預設值 `US (multiple regions in United States)`。
6. 點選「建立資料集」。
7. 確認資料集已建立，並列在「Explorer」窗格的專案 ID 下方。

您必須建立 BigQuery [連線](#)，才能與 Cloud Function 互動。如要建立遠端函式，必須先建立 BigQuery 連線。在本程式碼研究室中，我們將使用 BigLake 連線，透過 Cloud 函式從 BigQuery 存取模型。BigLake 連線有助於連結外部資料來源，同時保留精細的 BigQuery 存取控管和安全性，在本例中為 Vertex AI Gemini Pro API。

如要建立 **BigLake 連線**，請按照下列步驟操作：

1. 在 BigQuery 頁面的「Explorer」窗格中，按一下「新增」。



2. 按一下「連線至外部資料來源」。
3. 在「連線類型」清單中，選取「Vertex AI 遠端模型、遠端函式和 BigLake (Cloud 資源)」。
4. 在「連線 ID」欄位中，輸入連線名稱 `gemini-bq-conn`。
5. 將位置類型設為 `Multi-region`，並接受預設值 `US (multiple regions in United States)`。
6. 點選「建立連線」。
7. 按一下「前往連線」，然後複製「連線資訊」窗格中的服務帳戶 ID。

gemini-bq-conn QUERY SHARE

Connection info	
Connection ID	projects/[redacted]locations/us/connections/gemini-bq-conn
Friendly name	
Created	20 Mar 2024, 00:40:27 UTC+5:30
Last modified	20 Mar 2024, 00:40:27 UTC+5:30
Data location	us
Description	
Connection type	Vertex AI remote models, remote functions and BigLake (Cloud Resource)
Service account ID	bqcx-[redacted]@gcp-sa-bigquery-condel.iam.gserviceaccount.com

8. 前往「IAM 與管理」頁面，然後按一下「授予存取權」。
9. 將服務帳戶 ID 貼到「新增主體」欄位。
10. 從角色清單中選取 `Vertex AI user` 角色，然後按一下「儲存」。

Grant access to 'Abi's'

Resource

Abi's

Add principals

Principals are users, groups, domains or service accounts. [Learn more about principals in IAM](#)

New principals *

bqcx-...@gcp-sa-bigquery-condel.iam.gserviceaccount.com

Assign Roles

Roles are composed of sets of permissions and determine what the principal can do with this resource. [Learn more](#)

Role *

Vertex AI user

IAM condition (optional) ?

+ ADD IAM CONDITION

Grants access to use all resources in Vertex AI

+ ADD ANOTHER ROLE

SAVE

CANCEL

您已成功建立資料集和 BigQuery 連線。

6. 部署 Gemini 函式呼叫功能 (Java Cloud Function)

請按照下列步驟部署包含 Gemini 函式呼叫功能的 Java Cloud Function。

1. 使用下列指令，從 Cloud Shell 終端機複製 github [存放區](#)：

```
git clone https://github.com/AbiramiSukumaran/GeminiFunctionCalling
```

2. 將預留位置 `YOUR_API_KEY` 和 `YOUR_PROJECT_ID` 替換為您的值。

如果您已閱讀[這篇網誌](#)，就會知道函式呼叫實作項目使用[反向地理編碼 API](#)。請按照[這裡](#)的操作說明建立自己的 `API_KEY`。

3. 在 Cloud Shell 終端機中，前往新複製的專案目錄 **GeminiFunctionCalling**，然後執行下列陳述式，建構及部署 Cloud 函式：

```
gcloud functions deploy gemini-fn-calling --gen2 --region=us-central1 --runtime=java11 --source=. --entry-point=cloudcode.helloworld.HelloWorld --trigger-http
```

系統顯示「Allow unauthenticated invocations」(允許未經驗證的调用)問題時，請輸入「y」。建議您為企業應用程式設定驗證程序。但由於這是試用版應用程式，我們會繼續進行未經驗證的程序。

輸出內容為 REST 網址，格式如下：

`https://us-central1-YOUR_PROJECT_ID.cloudfunctions.net/gemini-fn-calling`

4. 如要測試這項 Cloud 函式，請在終端機中執行下列指令：

```
gcloud functions call gemini-fn-calling --region=us-central1 --gen2 --data '{"calls": [{"40.714224,-73.961452}"]}'
```

隨機範例提示的回應：

```
'{"replies":[{"DOOR_NUMBER": "277", "STREET_ADDRESS": "Bedford Ave", "AREA": null, "CITY": "Brooklyn", "TOWN": null, "COUNTY": "Kings County", "STATE": "NY", "COUNTRY": "USA", "ZIPCODE": "11211", "LANDMARK": null}]}'
```

這個 Cloud 函式的要求和回應參數實作方式與 BigQuery 的遠端函式调用相容。可直接從 BigQuery 資料就地取用。也就是說，如果您的資料輸入內容 (緯度和經度資料) 位於 BigQuery 中，您就可以對資料呼叫遠端函式，並取得函式回應，直接在 BigQuery 中儲存或處理。

5. 從 BigQuery 執行下列 DDL，建立可调用已部署 Cloud Functions 的遠端函式：

```
CREATE OR REPLACE FUNCTION
`mdm_gemini.MDM_GEMINI` (latlng STRING) RETURNS STRING
REMOTE WITH CONNECTION `us.gemini-bq-conn`
OPTIONS (
  endpoint = 'https://us-central1-YOUR_PROJECT_ID.cloudfunctions.net/gemini-fn-calling',
  max_batching_rows = 1
);
```

測試查詢，使用新建立的遠端函式：

```
SELECT mdm_gemini.MDM_GEMINI (latlong) from mdm_gemini.CITIBIKE_STATIONS limit 1;
```

如果使用 BigQuery 中建立的新遠端函式進行測試查詢時，因 Cloud Functions 權限問題而失敗，請前往 Google Cloud 控制台的 Cloud Functions，找出名為「**gemini-fn-calling**」的已部署 Cloud 函式。前往「權限」分頁，將主體新增為「allUsers」，並授予「Cloud Functions 叫用者」角色，確保所有使用者都能存取 Cloud Functions (僅限這個試用版應用程式)。

步驟 7 · 約 0 分鐘

7. 嘗試解決方法

如果您沒有反向地理編碼函式呼叫方法所需的 API_KEY，或因故未部署 Cloud Function，可以採取下列替代做法：

1. 從[存放區](#)下載 **CITIBIKE_STATIONS.csv** 檔案到 Cloud Shell 專案資料夾，然後前往該資料夾。
2. 在 Cloud Shell 終端機中執行下列指令，將 CSV 檔案中的資料匯出至新的 BigQuery 資料集 `mdm_gemini`：

```
bq load --source_format=CSV --skip_leading_rows=1 mdm_gemini.CITIBIKE_STATIONS
./CITIBIKE_STATIONS.csv \
name:string,latlng:string,capacity:numeric,num_bikes_available:numeric,num_docks_available:nu
meric,last_reported:timestamp,full_address_string:string
```

8. 建立資料表並擴充地址資料

步驟 1：建立表格

重要事項：如果您已使用解決方法，則必須已建立資料表，因此請略過這個步驟。

如果您尚未採用解決方法，請在 BigQuery SQL 編輯器中執行下列 DDL：

```
CREATE TABLE mdm_gemini.CITIBIKE_STATIONS as (  
  select  name, latitude || ',' || longitude as latlong, capacity, num_bikes_available,  
          num_docks_available,last_reported,  
          '' as full_address_string  
  from bigquery-public-data.new_york_citibike.citibike_stations) ;
```

現在，我們要在資料表中可用的經緯度座標上叫用遠端函式，藉此擴充地址資料。為資料設定下列條件：

- 2024 年回報
- 可供租借的自行車數量 > 0
- 容量大於 100

執行以下查詢：

```
update `mdm_gemini.CITIBIKE_STATIONS`  
set full_address_string = `mdm_gemini.MDM_GEMINI`(latlong)  
where EXTRACT(YEAR FROM last_reported) = 2024 and num_bikes_available > 0 and capacity > 100;
```

步驟 2：建立第二個來源，提供自行車站位置資料

即使您使用變通方法建立表格，也請勿略過這個步驟。

在本程式碼研究室中，您將在本步驟建立第二個來源，提供自行車租借站位置資料。這項作業是為了展示 MDM 如何整合多個來源的資料，並找出黃金真相。

在 BigQuery SQL 編輯器中執行下列 DDL，建立第二個位置資料來源，其中包含兩筆記錄。我們將這個資料表命名為 `mdm_gemini.CITIBIKE_STATIONS_SOURCE2`，並在其中插入兩筆記錄。

```
CREATE TABLE mdm_gemini.CITIBIKE_STATIONS_SOURCE2 (name STRING(55), address STRING(1000),  
embeddings_src ARRAY<FLOAT64>);  
  
insert into mdm_gemini.CITIBIKE_STATIONS_SOURCE2 VALUES ('Location broadway and 29',{  
  "DOOR_NUMBER": "1593", "STREET_ADDRESS": "Broadway", "AREA": null, "CITY": "New York",  
  "TOWN": null, "COUNTY": "New York County", "STATE": "NY", "COUNTRY": "USA", "ZIPCODE":  
  "10019", "LANDMARK": null},' null);  
  
insert into mdm_gemini.CITIBIKE_STATIONS_SOURCE2 VALUES ('Allen St & Hester',{  
  "DOOR_NUMBER": "36", "STREET_ADDRESS": "Allen St", "AREA": null, "CITY": "New York", "TOWN":  
  null, "COUNTY": "New York County", "STATE": "NY", "COUNTRY": "USA", "ZIPCODE": "10002",  
  "LANDMARK": null},' null);
```

9. 為地址資料生成嵌入

嵌入是指以高維度數值向量表示特定實體，例如一段文字或音訊檔案。機器學習 (ML) 模型會使用嵌入技術，將這類實體的語意編碼，方便進行推論和比較。舉例來說，在叢集、分類和建議模型中，常見的作業是測量嵌入空間中向量之間的距離，找出語意上最相似的項目。Vertex AI 文字嵌入 API 可讓您使用 Vertex AI 的生成式 AI 建立文字嵌入。文字嵌入是文字的數值表示法，可擷取字詞和詞組之間的關係。如要進一步瞭解 Vertex AI Text Embeddings，請參閱[這篇文章](#)。

1. 執行下列 DDL，為 Vertex AI 文字嵌入 API 建立遠端模型：

```
CREATE OR REPLACE MODEL `mdm_gemini.CITIBIKE_STATIONS_ADDRESS_EMB`  
REMOTE WITH CONNECTION `us.gemini-bq-conn`  
OPTIONS (ENDPOINT = 'textembedding-gecko@latest');
```

2. 遠端嵌入模型準備就緒後，請使用下列查詢，為第一個來源生成嵌入項目並儲存在資料表中：

```
CREATE TABLE `mdm_gemini.CITIBIKE_STATIONS_SOURCE1` AS (  
  SELECT *  
  FROM ML.GENERATE_EMBEDDING(  
    MODEL `mdm_gemini.CITIBIKE_STATIONS_ADDRESS_EMB`,  
    ( select name, full_address_string as content from `mdm_gemini.CITIBIKE_STATIONS`  
      where full_address_string is not null )  
  )  
);
```

您也可以將嵌入結果欄位儲存在先前建立的相同 **mdm_gemini.CITIBIKE_STATIONS** 資料表中，不必建立新資料表。

3. 如要為 **CITIBIKE_STATIONS_SOURCE2** 資料表中的地址資料生成嵌入，請執行下列查詢：

```
update `mdm_gemini.CITIBIKE_STATIONS_SOURCE2` a set embeddings_src =  
(  
  SELECT ml_generate_embedding_result  
  FROM ML.GENERATE_EMBEDDING(  
    MODEL `mdm_gemini.CITIBIKE_STATIONS_ADDRESS_EMB`,  
    ( select name, address as content from `mdm_gemini.CITIBIKE_STATIONS_SOURCE2` ))  
  where name = a.name) where name is not null;
```

這應該會為第二個來源建立嵌入內容。請注意，我們已在同一個資料表 **CITIBIKE_STATIONS_SOURCE2** 中建立嵌入欄位。

4. 如要將來源資料表 1 和 2 產生的嵌入內容視覺化，請執行下列查詢：

```
select name,address,embeddings_src from `mdm_gemini.CITIBIKE_STATIONS_SOURCE2`;  
select name,content,ml_generate_embedding_result from `mdm_gemini.CITIBIKE_STATIONS_SOURCE1`;
```

現在，請執行向量搜尋來找出重複項目。

10. 執行向量搜尋，標記重複的地址

在這個步驟中，您會搜尋 `mdm_gemini.CITIBIKE_STATIONS_SOURCE1` 資料表的地址嵌入 `ml_generate_embedding_result` 資料欄，找出與 `mdm_gemini.CITIBIKE_STATIONS_SOURCE2` 資料表 `embeddings_src` 資料欄中每個資料列最相符的前兩個嵌入。

如要執行這項操作，請執行下列查詢：

```
select query.name name1,base.name name2,
/* (select address from mdm_gemini.CITIBIKE_STATIONS_SOURCE2 where name = query.name)
content1, base.content content2, */
distance
from VECTOR_SEARCH(
  TABLE mdm_gemini.CITIBIKE_STATIONS_SOURCE1,
  'ml_generate_embedding_result',
  (SELECT * FROM mdm_gemini.CITIBIKE_STATIONS_SOURCE2),
  'embeddings_src',
  top_k => 2
) where query.name <> base.name
order by distance desc;
```

要查詢的資料表：`mdm_gemini.CITIBIKE_STATIONS_SOURCE1` 欄位 `ml_generate_embedding_result`

做為基礎的資料表：`mdm_gemini.CITIBIKE_STATIONS_SOURCE2` 位於欄位 `embeddings_src`

top_k:指定要傳回的最近鄰點數量。預設值為 10。負值會視為無限大，也就是說，所有值都會計為鄰近值並傳回。

distance_type:指定用於計算兩個向量之間距離的指標類型。支援的距離類型包括歐幾里得和餘弦。預設值為「歐幾里得」。

查詢結果如下：

Row	name1	name2	distance
1	Allen St & Hester	Broadway & E 14 St	0.37209942848324246
2	Location broadway and 29	Broadway & E 14 St	0.18214325356002756
3	Location broadway and 29	Broadway & W 48 St	0.00677691544653693
4	Allen St & Hester	Allen St & Hester St	0.0054761370041464365

如您所見，這會列出 `CITIBIKE_STATIONS_SOURCE2` 中兩列的兩個最鄰近鄰點 (也就是最接近的重複項目)。`CITIBIKE_STATIONS_SOURCE1` 由於 `distance_type` 未指定，因此系統會假設為歐幾里得，並將距離解讀為兩個來源之間地址 TEXT 值的距離，最低者為最相似的地址文字。

讓我們使用下列查詢，將 `distance_type` 設為 **Cosine**：

```

select query.name name1,base.name name2,
/* (select address from mdm_gemini.CITIBIKE_STATIONS_SOURCE2 where name = query.name)
content1, base.content content2, */
distance
from VECTOR_SEARCH(
TABLE mdm_gemini.CITIBIKE_STATIONS_SOURCE1,
'ml_generate_embedding_result',
(SELECT * FROM mdm_gemini.CITIBIKE_STATIONS_SOURCE2),
'embeddings_src',
top_k => 2,distance_type => 'COSINE'
) where query.name <> base.name
order by distance desc;

```

查詢結果如下：

Row	name1	name2	distance
1	Allen St & Hester	Broadway & E 14 St	0.069229283613...
2	Location broadway and 29	Broadway & E 14 St	0.016588153745...
3	Location broadway and 29	Broadway & W 48 St	2.296339143703...
4	Allen St & Hester	Allen St & Hester St	1.499410095484...

這兩項查詢 (兩種距離類型) 都會依距離遞減排序，也就是說，我們要依距離遞減順序列出結果。但你會發現第二個查詢的距離順序已反轉。你能猜到原因嗎？

Yes!! 你答對了！在餘弦相似度中，數字越大表示相似度越高，距離越小。在歐氏距離中，數字越大表示值之間的距離越遠。

如要進一步瞭解 MDM，以及歐幾里得和餘弦距離的差異與應用，請參閱[這篇網誌](#)。

11. 清除所用資源

如要避免系統向您的 Google Cloud 帳戶收取本文章所用資源的費用，請按照下列步驟操作：

1. 在 Google Cloud 控制台中前往「管理資源」頁面。
 2. 在專案清單中選取要刪除的專案，然後點按「刪除」。
 3. 在對話方塊中輸入專案 ID，然後按一下「Shut down」(關機) 即可刪除專案。
 4. 如要保留專案，請略過上述步驟，然後前往 Cloud Functions，在函式清單中勾選要刪除的函式，並按一下「刪除」。
-

12. 恭喜

恭喜！您已展現使用 Gemini 1.0 Pro 和函式呼叫的強大功能，將幾項 MDM 活動轉換為簡化但功能強大、具決定性且可靠的生成式 AI 功能。現在您已瞭解這項功能，歡迎找出其他實作相同用途或 MDM 功能的方式。您是否可以驗證資料集、填補資訊缺口，或透過內嵌在生成式 AI 回覆中的結構化呼叫，自動執行工作？如需更深入的指引，請參閱 [Vertex AI](#)、[BigQuery 遠端函式](#)、[Cloud Functions](#)、[Embeddings](#) 和 [Vector Search](#) 的說明文件。這是本專案的 GitHub [存放區](#)。歡迎與我們分享您運用所學知識打造的內容！
